

Weighted Interlaced Caching Lower Bound

Tal Zohar

April 5, 2026

1 The Varying Cache Size Problem

Consider the following variant of weighted paging. The page requests are known in an offline manner, however the cache size changes in an online manner.

Every request is represented as a tuple (p_t, k_t) where p_t is the page number and k_t is the size of the cache upon request.

Let σ be the page request sequence and let ϕ be the cache size request sequence.

Define The Restricted Varying Cache Size Problem (RVCSP) as the Varying Cache Size Problem but the cache size is restricted to take only values k and $k - 1$.

Lemma 1. *If there exists an $\Omega(\alpha)$ deterministic / randomized lower bound for RVCSP, then there exists an $\Omega(\alpha)$ deterministic / randomized lower bound for The Interlaced caching problem for the case where $l = 2$.*

Proof. Given some (σ, ϕ) RVCSP instance (where σ is known offline and ϕ is given online), we can transform it into an instance of The Interlaced Caching Problem with 2 processes P_1, P_2 . P_1, P_2 have request sequences σ_1, σ_2 respectively, defined as follows. Set σ_1 to σ and set σ_2 to $q, q, q, \dots$ where q is a page of weight ϵ . Now, schedule the ICP instance as follows. Upon each request of the RVCSP instance:

1. If cache size is $k - 1$ schedule P_2 for a sufficiently large amount of requests.
2. Schedule P_1 once so that σ_1 follows σ .

□

Similarly to the lower bound for The Parking Permit Problem, we can provide an $\Omega(k)$ deterministic lower bound and an $\Omega(\log k)$ randomized lower bound for RVCSP.

2 RVCSP $\Omega(k)$ Deterministic Lower Bound

Define $M = 2k$.

Define $c_j = 2^{j-1}$ for pages $p_1, p_2, \dots, p_k$. Define σ_k recursively as follows.

$$\sigma_j = \begin{cases} p_j \overbrace{\sigma_{j-1} \sigma_{j-1} \dots \sigma_{j-1}}^{M \text{ times}} p_j & j > 1 \\ p_1 & j = 1 \end{cases}$$

We will show that given page request sequence σ , for every algorithm ALG there exists a cache size sequence ϕ such that $ALG(\sigma, \phi) \geq k \cdot OPT(\sigma, \phi)$.

Define ϕ so that ALG never has k pages in its cache. This means, whenever ALG has $k - 1$ pages in its cache, and there is a request for a page not in the cache, set the cache size to $k - 1$. Otherwise set the cache size to k .

Observe that the request sequence σ_k contains many subsequences of type σ_j . We say that the amount ALG pays on some subsequence σ_j is the sum of page intervals he discarded that are fully contained within that subsequence.

We call subsequence σ_j targeted if it contains an instance where the cache size is $k - 1$. For every targeted subsequence σ_j we can distinguish between three cases:

1. ALG removes some p_i from the cache for $i > j$.
2. Case 1 invalid and ALG removes p_j from the cache.
3. Case 1 and 2 invalid (In this case, for every cache size reduction, ALG removes some p_i from the cache for $i < j$)

Denote the amount of times case 2 occurs for subsequences σ_j by n_j . Denote the amount of times case 3 occurs for subsequences σ_j by r_j .

Note that the amount of times case 1 occurs for subsequences σ_j is $\sum_{i > k} n_i$, since each instance of case 1 for some subsequence σ_j can be matched with an instance of case 2 for some containing subsequence σ_i .

This gives us the following lower bound $c(ALG) \geq \sum_{j=1}^k c_j \cdot n_j$.

Lemma 2. *For every targeted subsequence σ_j of case 2, ALG pays at least c_j . For every targeted subsequence σ_j of case 3, ALG pays at least $k \cdot c_j$.*

Proof. We prove inductively. The first part of the lemma is immediate. For the second part, apply the inductive hypothesis on every of the M σ_{j-1} instances. We know that each of the subsequences must be targeted of case 2 or 3, since ALG must keep all p_i 's for $i > j$ in cache throughout. Thus, by applying the inductive hypothesis, we know that ALG pays at least $M \cdot c_{j-1} = k \cdot c_j$. $\square$

The lemma gives us the following lower bound $c(ALG) \geq r_j c_j$ for every j .

Now, consider the algorithm which upon eviction evicts p_j if in cache, otherwise evicts p_1 . We can view σ as a sequence of σ_j separated by requests for pages p_i for $i > j$. If σ_j is targeted, then the algorithm must evict p_j and then p_1 on its second request. Otherwise, it does nothing. Thus, the algorithm pays $c_j + c_1$ times the amount of targeted subsequences σ_j . This yields the following upper bound on OPT

$$c(OPT) \leq (c_j + c_1) \left(r_j + \sum_{i \geq j} n_i \right)$$

Putting it together

$$\begin{aligned}
k \cdot c(OPT) &\leq \sum_{j=1}^k (c_j + 0) \left(r_j + \sum_{i \geq j} n_i \right) \\
&= \sum_{j=1}^k \left(n_j \sum_{i=1}^j c_i + r_j \cdot c_j \right) \\
&< \sum_{j=1}^k (2 \cdot c_j \cdot n_j + r_j c_j) < 2 \cdot c(ALG) + c(ALG) = 3 \cdot c(ALG)
\end{aligned}$$